



VISUAL PROGRAMMING

Introduction to GUI programming

MUSBAH J. MOUSA
IYAS A. I. ESHAIKHKHALIL

COURSE DESCRIPTION

- **Programming applications with graphical user interfaces.**
- **Topics include controls, events, resources, data bindings, animations and styles.**
- **Students will go through the process of building desktop applications from start to finish using Java.**
- **You will learn how to leverage your Java programming knowledge and explore JavaFX libraries to design graphical interfaces, simple animations, and mini-games.**
- **Students will learn how to connect Java applications to databases.**

COURSE LEARNING OUTCOMES

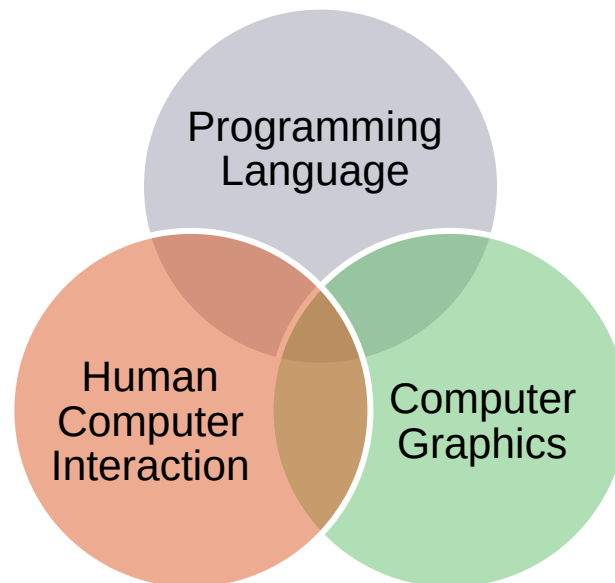
- 1. Building and strengthening programming and software design skills.**
- 2. Understand and apply basic user interface design principles.**
- 3. Understand and learn to use the available APIs and tools to build and design: animations, games, and user interfaces.**
- 4. Learn how to link software to databases.**
- 5. Building an integrated application that contains user interfaces and connects to a database.**

OUTLINES

- Visual programming
- JavaFX vs Swing and AWT.
- What is JavaFX?
- Features of JavaFX.
- History of JavaFX.
- JavaFX Application Structure.
- Lifecycle of JavaFX Application.
- First JavaFX Application

VISUAL PROGRAMMING

- **GUI (Graphical User Interface)** is a Graphical Interface that is a visual representation of communication presented to the user for easy interaction with the machine.
- It is the common user Interface that includes Graphical representation like buttons and icons, and communication can be performed by interacting with these icons rather than the usual text-based or command-based communication.



VISUAL PROGRAMMING

LANGUAGES



FRAMEWORKS



Windows Forms

WPF



JAVAFX VS SWING AND AWT

- **Swing** and **AWT** are replaced by the **JavaFX** platform for developing rich Internet applications.
- When Java was introduced, the GUI classes were bundled in a library known as the **Abstract Windows Toolkit (AWT)**.
- **AWT** is fine for developing simple graphical user interfaces, but not for developing comprehensive GUI projects. In addition, **AWT** is prone to platform-specific bugs.
- The **AWT** user-interface components were replaced by a more robust, versatile, and flexible library known as **Swing** components.
- **Swing** components are painted directly on canvases using Java code.
- **Swing** components depend less on the target platform and use less of the native GUI resource.
- With the release of Java 8, **Swing** is replaced by a completely new GUI platform known as **JavaFX**.

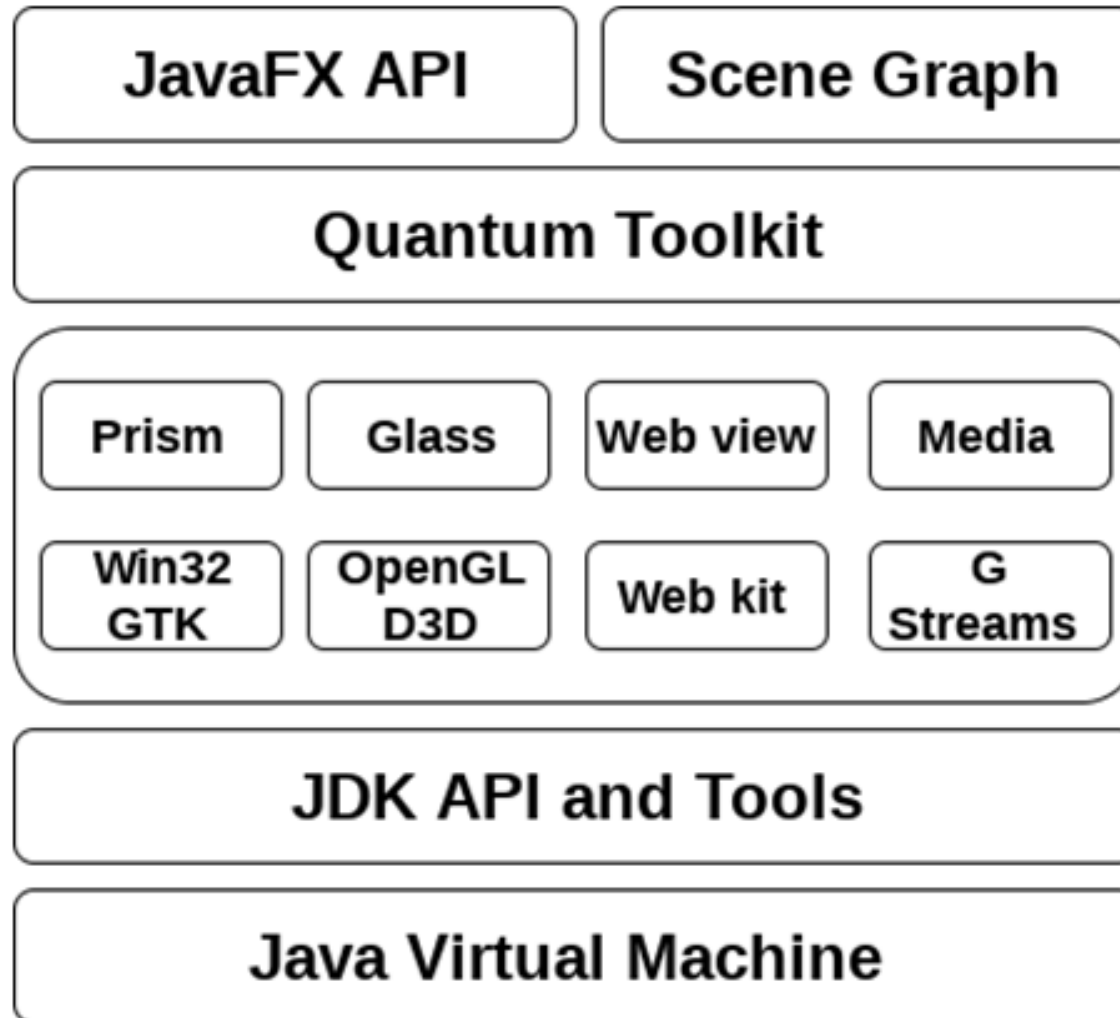
WHAT IS JAVAFX?

- JavaFX is a Java library used to develop Desktop applications as well as Rich Internet Applications (RIA). The applications built in JavaFX, can run on multiple platforms including Web, Mobile and Desktops.
- JavaFX is intended to replace swing in Java applications as a GUI framework. However, It provides more functionalities than swing.
- Like Swing, JavaFX also provides its own components and doesn't depend upon the operating system. It is lightweight and hardware accelerated. It supports various operating systems including Windows, Linux and Mac OS.

FEATURES OF JAVA FX

- It is written in JAVA. This rich technology comes under Java toolkit version 8 or above.
- This technology supports CSS styling.
- It provides a JavaScript engine.
- We can have a 2D/3D graphics object added to our applications.
- It supports image, media format (mp3, aiff) and video (flv).
- It has a WebView so that HTML components as well can be added with JavaFX.
- Since it is built on top of swing so we can use swing features as well in JavaFX.
- It supports the use of XML to have separate data structures differently maintained from application code.
- You can avoid hassles of browser dependent errors of HTML, performance issues with the richness of JavaFX technology.

JAVAFX – ARCHITECTURE



JAVAFX – ARCHITECTURE

- **JavaFX public API**

- The top layer of JavaFX architecture contains a JavaFX public API which provides all the necessary classes that are responsible for executing a full featured JavaFX application.

- **Scene Graph**

- It is the starting point of constructing a JavaFX application. It is a hierarchical tree of nodes that represent all the visual elements of user interface. It also have the capability of handling event. In general, scene graph can be defined as a collection of nodes.
- Each node has its separate id, style and volume. Every node of a scene graph can only have single parent and zero or more children.

JAVAFX – ARCHITECTURE

- **Graphics Engine**
 - The JavaFX graphics engine provides the graphics support to the scene graph. It basically supports 2D as well as 3D graphics both. It provides the software rendering when the graphics hardware present on the system is not able to support hardware accelerated rendering.
- **The two graphics accelerated pipelines in the JavaFX are:**
 1. Prism
 - prism can be seen as High Performance hardware-accelerated graphics pipeline. It has the capability to render both 2D and 3D graphics. Prism implements different ways to render graphics on different platforms.
 - DirectX 9 on windows XP or vista
 - DirectX 11 on windows 7
 - OpenGL on Mac, Linux and embedded
 - Java 2D when hardware acceleration is not possible

JAVAFX – ARCHITECTURE

2. Quantum Tool kit

- Quantum Tool Kit is used to bind prism and glass windowing tool kit together and makes them available for the above layers in stack.

- **Glass Windowing tool kit**

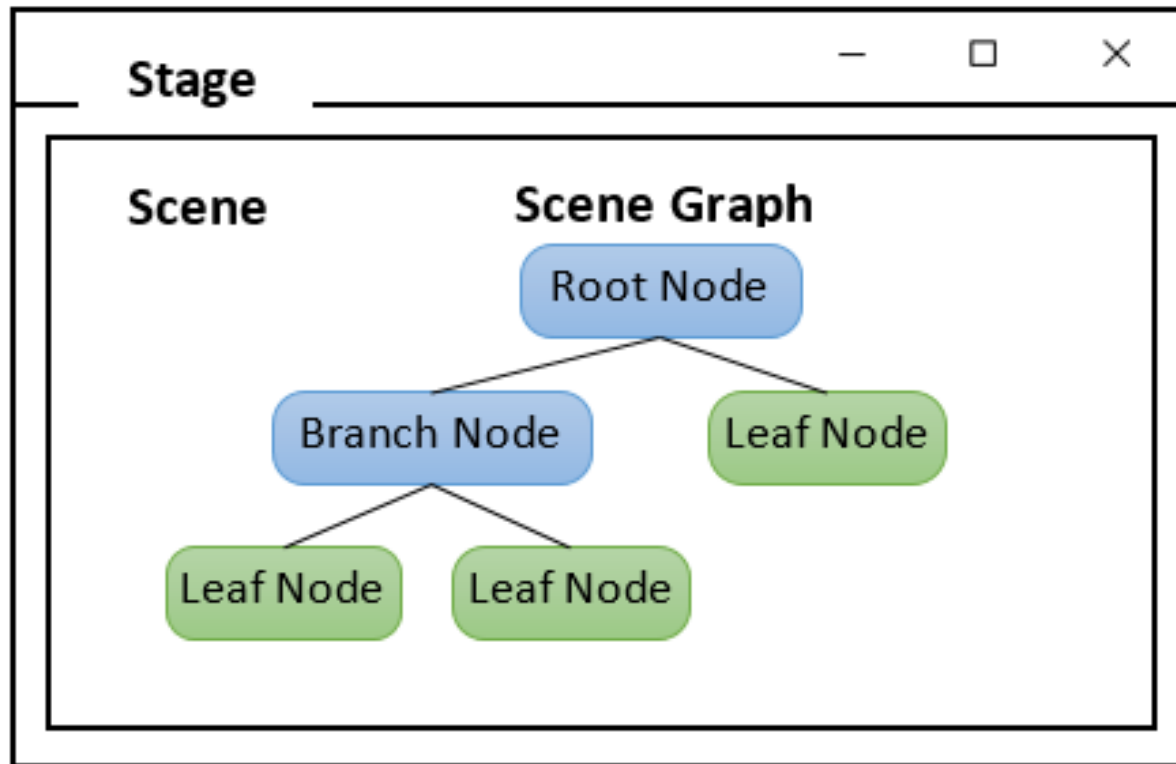
- It is present on the lowest level of JavaFX graphics stack. It basically can be seen as a platform dependent layer which works as an interface between JavaFX platform and native operating system.
- It is responsible for providing the operating system services such as managing the windows, timers, event queues and surfaces.

JAVAFX – ARCHITECTURE

- **Web View**
 - We can also embed the HTML content to a JavaFX scene graph. For this purpose, JavaFX uses a component called web view. Web view uses web kit which is an internal open source browser and can render HTML5, DOM, CSS, SVG and JavaScript.
- **Media Engine**
 - By using Media engine, the JavaFX application can support the playback of audio and video media files. JavaFX media engine depends upon an open source engine called as G Streamer. The package `javafx.scene.media` contains all the classes and interfaces that can provide media functionalities to JavaFX applications.

JAVAFX APPLICATION STRUCTURE

- In general, a JavaFX application will have three major components namely Stage, Scene and Nodes as shown in the following diagram.



JAVAFX APPLICATION STRUCTURE

- **Stage**
 - A stage (a window) contains all the objects of a JavaFX application. It is represented by Stage class of the package javafx.stage. You have to call the show() method to display the contents of a stage.
- **Scene**
 - Scene actually holds all the physical contents (nodes) of a JavaFX application. Javafx.scene.Scene class provides all the methods to deal with a scene object. Creating scene is necessary in order to visualize the contents on the stage.

JAVAFX APPLICATION STRUCTURE

- **Scene graph**
 - A scene graph is a data structure similar to a tree, in modern graphical applications, it is a collection of nodes. In a JavaFX application the `javafx.scene`. The scene class holds all the contents of a scene graph.
 - While creating a scene it is mandatory to specify the root node.
 - In an instance, the scene object is added to only one stage.

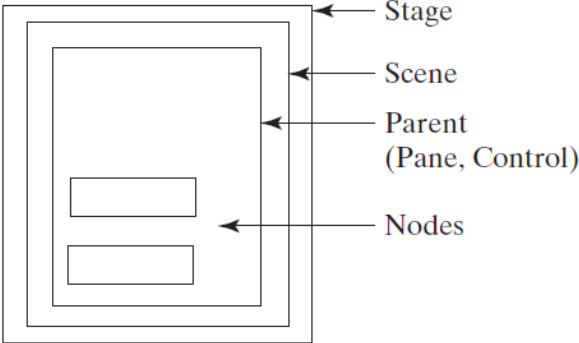
JAVAFX APPLICATION STRUCTURE

- **Nodes**
 - A node is a visual/graphical primitive object of a JavaFX application.
 - 2D and 3D geometric objects (circle, sphere, etc.), UI control objects (button, checkbox, etc.), Container/layout objects (Border Pane, Grid Pane, etc.), media element objects (audio, video, image objects, etc.) are (some) examples of nodes in JavaFX.
 - Each node in the scene graph has a single parent, and the node which does not contain any parents is known as the root node.
 - Every parent node has one or more children, and the node without children is termed as leaf node; a node with children is termed as a branch node.

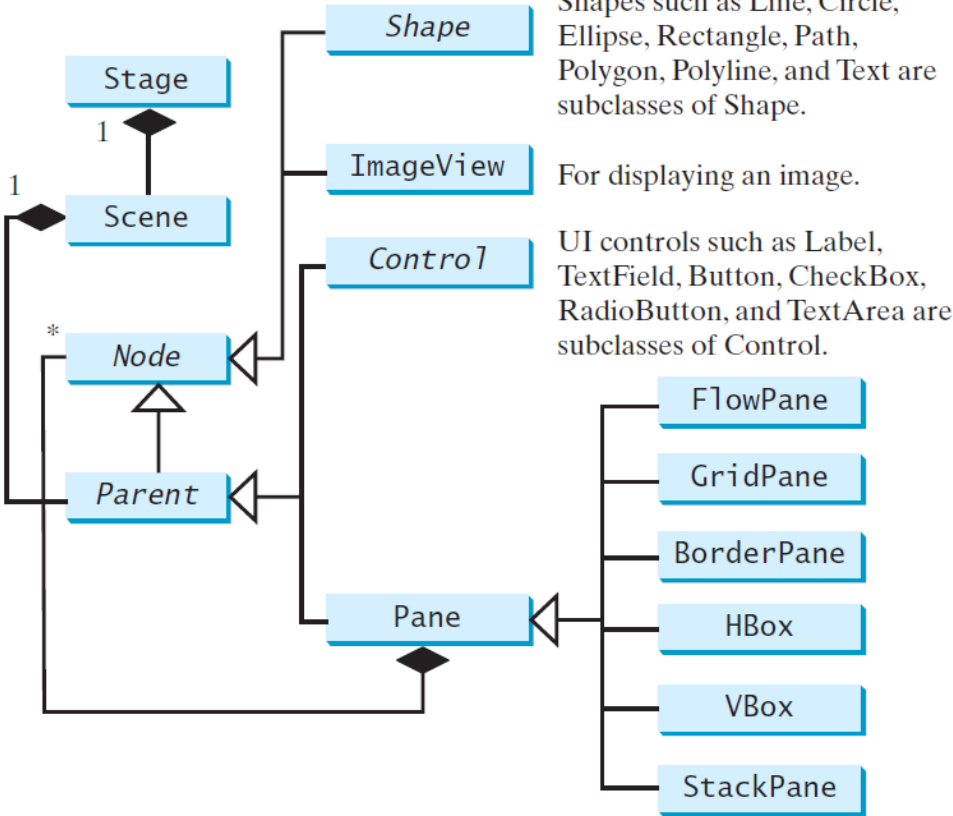
61

MyJavaFX

MultipleStageDemo



(a)



(b)

JAVAFX APPLICATION STRUCTURE

- The following are various kinds of root nodes in a JavaFX application.
 - **Group** – A group node is a collective node that contains a list of children nodes. Whenever the group node is rendered, all its child nodes are rendered in order. Any transformation, effect state applied to the group will be applied to all the child nodes.
 - **Region** – It is the base class of all the JavaFX Node-based UI Controls, such as Chart, Pane and Control.
 - **WebView** – This node manages the web engine and displays its contents.
- **Leaf Node** – The node without child nodes is known as the leaf node. For example, Rectangle, Ellipse, Box, ImageView, MediaView are examples of leaf nodes.

LIFECYCLE OF JAVAFX APPLICATION

- The JavaFX Application class has three life cycle methods, which are –
 - **start()** – The entry point method where the JavaFX graphics code is to be written.
 - **stop()** – An empty method which can be overridden, here you can write the logic to stop the application.
 - **init()** – An empty method which can be overridden, but you cannot create a stage or scene in this method.

LIFECYCLE OF JAVAFX APPLICATION

- In addition to these, it provides a static method named **launch()** to launch JavaFX application.
- Whenever a JavaFX application is launched, the following actions will be carried out (in the same order).
 - An instance of the application class is created.
 - Init() method is called.
 - The start() method is called.
 - The launcher waits for the application to finish and calls the stop() method.

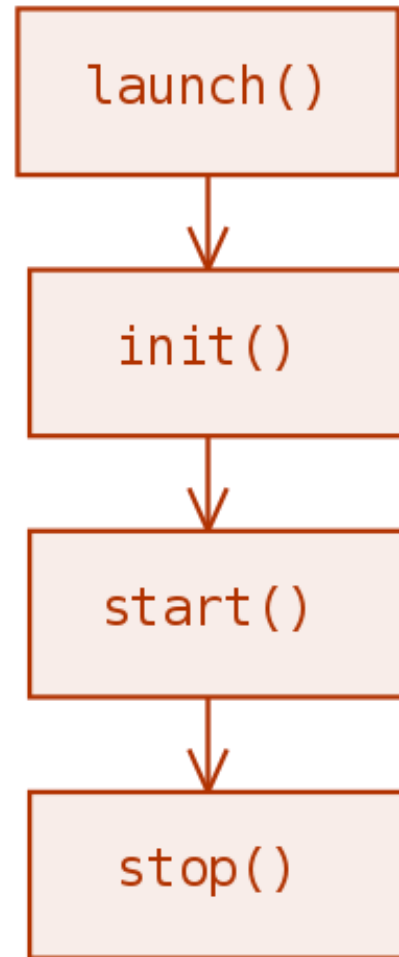
LIFECYCLE OF JAVAFX APPLICATION

- The main method is not needed if you run the program from the command line.
- It may be needed to launch a JavaFX program from an IDE with a limited JavaFX support.
- When you run a JavaFX application without a main method, JVM automatically invokes the launch method to run the application.

LIFECYCLE OF JAVAFX APPLICATION

- **Terminating the JavaFX Application**
 - When the last window of the application is closed, the JavaFX application is terminated implicitly.
 - You can turn this behavior off bypassing the Boolean value “False” to the static method `setImplicitExit()` (should be called from a static context).
 - You can terminate a JavaFX application explicitly using the methods `Platform.exit()` or `System.exit(int)`.

LIFECYCLE OF JAVAFX APPLICATION



FIRST JAVAFX APPLICATION

- **Step 1: Extend javafx.application.Application and override start()**
 - As we have studied earlier that **start()** method is the starting point of constructing a JavaFX application therefore we need to first override start method of **javafx.application.Application** class. Object of the class **javafx.stage.Stage** is passed into the **start()** method therefore import this class and pass its object into start method. **JavaFX.application.Application** needs to be imported in order to override start method.

```
package application;
import javafx.application.Application;
import javafx.stage.Stage;
public class Hello_World extends Application{

    @Override
    public void start(Stage primaryStage) throws Exception {
        // TODO Auto-generated method stub

    }

}
```

FIRST JAVAFX APPLICATION

- **Step 2: Launch the application**
 - We will call the `launch()` method and pass the command line arguments (`args`) in main method.

```
/**
 * @param args the command line arguments
 */
public static void main(String[] args) {
    launch(args);
}
```

FIRST JAVAFX APPLICATION

- **Step 3: Create a Button**
 - A button can be created by instantiating the `javafx.scene.control.Button` class. For this, we have to import this class into our code. Pass the button label text in `Button` class constructor.

```
package application;
import javafx.application.Application;
import javafx.scene.control.Button;
import javafx.stage.Stage;
public class Hello_World extends Application{

    @Override
    public void start(Stage primaryStage) throws Exception {
        // TODO Auto-generated method stub
        Button btn1=new Button( string:"Say, Hello World");

    }

}
```

FIRST JAVAFX APPLICATION

- **Step 4: Create a layout and add button to it**
 - JavaFX provides the number of layouts. We need to implement one of them in order to visualize the widgets properly. It exists at the top level of the scene graph and can be seen as a root node. All the other nodes (buttons, texts, etc.) need to be added to this layout.

```
package application;
import javafx.application.Application;
import javafx.scene.control.Button;
import javafx.stage.Stage;
import javafx.scene.layout.StackPane;
public class Hello_World extends Application{

    @Override
    public void start(Stage primaryStage) throws Exception {
        // TODO Auto-generated method stub
        Button btn1=new Button( string:"Say, Hello World");
        StackPane root=new StackPane();
        root.getChildren().add( e:btn1);

    }

}
```

FIRST JAVAFX APPLICATION

- **Step 5: Create a Scene**
 - The layout needs to be added to a scene. Scene remains at the higher level in the hierarchy of application structure. It can be created by instantiating `javafx.scene.Scene` class. We need to pass the layout object to the scene class constructor.

```
package application;
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.stage.Stage;
import javafx.scene.layout.StackPane;
public class Hello_World extends Application{

    @Override
    public void start(Stage primaryStage) throws Exception {
        // TODO Auto-generated method stub
        Button btn1=new Button( string:"Say, Hello World");
        StackPane root=new StackPane();
        root.getChildren().add( e:btn1);
        Scene scene=new Scene( parent:root);
    }
}
```

FIRST JAVAFX APPLICATION

- **Step 6: Prepare the Stage**
 - javafx.stage.Stage class provides some important methods which are required to be called to set some attributes for the stage. We can set the title of the stage. We also need to call show() method without which, the stage won't be shown.

```
package application;
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.stage.Stage;
import javafx.scene.layout.StackPane;
public class Hello_World extends Application{

    @Override
    public void start(Stage primaryStage) throws Exception {
        // TODO Auto-generated method stub
        Button btn1=new Button( string:"Say, Hello World");
        StackPane root=new StackPane();
        root.getChildren().add( e:btn1);
        Scene scene=new Scene( parent:root);
        primaryStage.setScene( value: scene);
        primaryStage.setTitle( value: "First JavaFX Application");
        primaryStage.show();
    }
}
```

FIRST JAVAFX APPLICATION

- **Step 7: Create an event for the button**
 - As our application prints hello world for an event on the button. We need to create an event for the button. For this purpose, call `setOnAction()` on the button and define a anonymous class `EventHandler` as a parameter to the method.

```
btn1.setOnAction(new EventHandler<ActionEvent>() {  
  
    @Override  
    public void handle(ActionEvent arg0) {  
        // TODO Auto-generated method stub  
        System.out.println("hello world");  
    }  
});
```

ButtonInPane

THE COLOR CLASS

JavaFX defines the abstract Paint class for painting a node.

The `javafx.scene.paint.Color` is a concrete subclass of Paint , which is used to encapsulate colors.

THE COLOR CLASS

The getter methods for property values are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.paint.Color

-red: double
-green: double
-blue: double
-opacity: double

+Color(r: double, g: double, b: double, opacity: double)
+brighter(): Color
+darker(): Color
+color(r: double, g: double, b: double): Color
+color(r: double, g: double, b: double, opacity: double): Color
+rgb(r: int, g: int, b: int): Color
+rgb(r: int, g: int, b: int, opacity: double): Color

The red value of this Color (between 0.0 and 1.0).

The green value of this Color (between 0.0 and 1.0).

The blue value of this Color (between 0.0 and 1.0).

The opacity of this Color (between 0.0 and 1.0).

Creates a Color with the specified red, green, blue, and opacity values.

Creates a Color that is a brighter version of this Color.

Creates a Color that is a darker version of this Color.

Creates an opaque Color with the specified red, green, and blue values.

Creates a Color with the specified red, green, blue, and opacity values.

Creates a Color with the specified red, green, and blue values in the range from 0 to 255.

Creates a Color with the specified red, green, and blue values in the range from 0 to 255 and a given opacity.

THE COLOR CLASS

- A color instance can be constructed using the following constructor:
- ***public Color(double r, double g, double b, double opacity);***
- in which r , g , and b specify a color by its red, green, and blue components with values in the range from 0.0 (darkest shade) to 1.0 (lightest shade).
- The opacity value defines the transparency of a color within the range from 0.0 (completely transparent) to 1.0 (completely Opaque).
- This is known as the RGBA model, where RGBA stands for red, green, blue, and alpha. The alpha value indicates the opacity.

THE COLOR CLASS

- The Color class is immutable. Once a Color object is created, its properties cannot be changed. The `brighter()` method returns a new Color with a larger red, green, and blue values, and the `darker()` method returns a new Color with a smaller red, green, and blue values.

THE FONT CLASS

- **A Font is defined by its name, weight, posture, and size.**
 - Times New Roman, Courier, and Arial are examples of font names.
- **You can obtain a list of available font family names by invoking the static `getFontNames()` method. This method returns `List<String>`.**
- **There are two font postures defined as constants in the `FontPosture` class:**
 - `FontPosture.ITALIC`
 - `FontPosture.REGULAR`
- **A Font object is immutable. Once a Font object is created, its properties cannot be change**

THE FONT CLASS

javafx.scene.text.Font

-size: double
-name: String
-family: String

+Font(size: double)
+Font(name: String, size: double)
+font(name: String, size: double)
+font(name: String, w: FontWeight, size: double)
+font(name: String, w: FontWeight, p: FontPosture, size: double)
+getFamilies(): List<String>
+getFontNames(): List<String>

The getter methods for property values are provided in the class, but omitted in the UML diagram for brevity.

The size of this font.

The name of this font.

The family of this font.

Creates a **Font** with the specified size.

Creates a **Font** with the specified full font name and size.

Creates a **Font** with the specified name and size.

Creates a **Font** with the specified name, weight, and size.

Creates a **Font** with the specified name, weight, posture, and size.

Returns a list of font family names.

Returns a list of full font names including family and weight.

THE FONT CLASS

javafx.scene.text.Font

-size: double
-name: String
-family: String

+Font(size: double)
+Font(name: String, size: double)
+font(name: String, size: double)
+font(name: String, w: FontWeight, size: double)
+font(name: String, w: FontWeight, p: FontPosture, size: double)
+getFamilies(): List<String>
+getFontNames(): List<String>

The getter methods for property values are provided in the class, but omitted in the UML diagram for brevity.

The size of this font.

The name of this font.

The family of this font.

Creates a **Font** with the specified size.

Creates a **Font** with the specified full font name and size.

Creates a **Font** with the specified name and size.

Creates a **Font** with the specified name, weight, and size.

Creates a **Font** with the specified name, weight, posture, and size.

Returns a list of font family names.

Returns a list of full font names including family and weight.

IMAGE AND IMAGEVIEW

- The **`javafx.scene.image.Image`** class represents a graphical image and is used for loading an image from a specified filename or a URL.
- For example,

`new Image("image/us.gif")`

creates an Image object for the image file `us.gif` under the directory `image` in the Java class directory

and

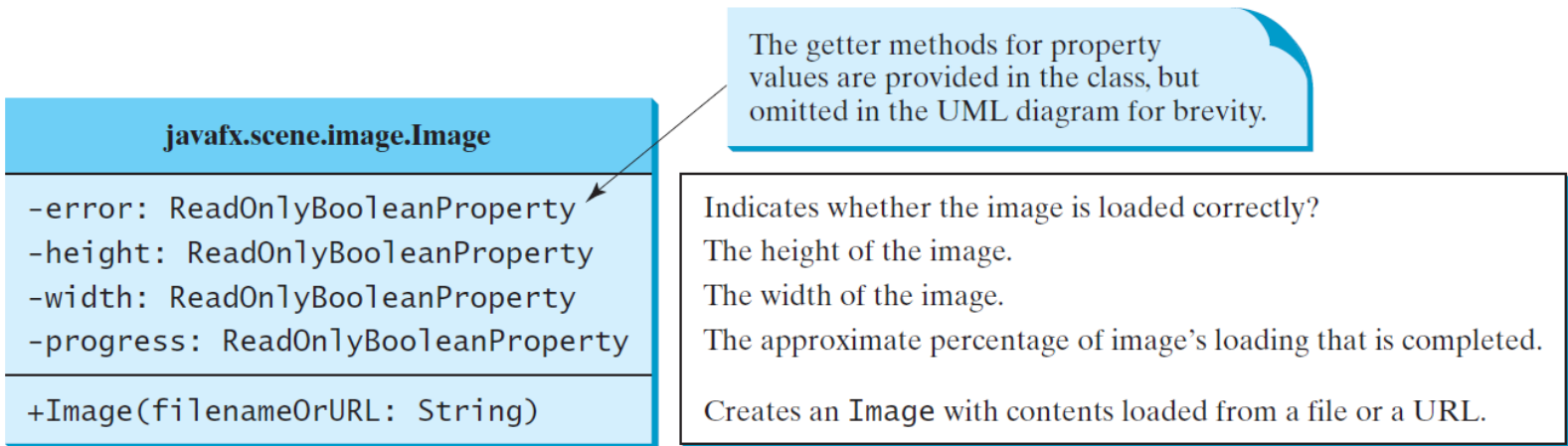
`new Image("http://liveexample.pearsoncmg.com/book/image/us.gif")`

creates an Image object for the image file in the URL on the Web.

IMAGE AND IMAGEVIEW

- The `javafx.scene.image.ImageView` is a node for displaying an image. An `ImageView` can be created from an `Image` object.
- **Note an `Image` object can be shared by multiple nodes. In this case, it is shared by three `ImageView`. However, a node such as `ImageView` cannot be shared. You cannot place an `ImageView` multiple times into a pane or scene.**

THE IMAGE CLASS



THE IMAGEVIEW CLASS

`javafx.scene.image.ImageView`

`-fitHeight: DoubleProperty`
`-fitWidth: DoubleProperty`
`-x: DoubleProperty`
`-y: DoubleProperty`
`-image: ObjectProperty<Image>`

`+ImageView()`
`+ImageView(image: Image)`
`+ImageView(filenameOrURL: String)`

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The height of the bounding box within which the image is resized to fit.
The width of the bounding box within which the image is resized to fit.
The x-coordinate of the ImageView origin.
The y-coordinate of the ImageView origin.
The image to be displayed in the image view.

Creates an `ImageView`.
Creates an `ImageView` with the specified image.
Creates an `ImageView` with image loaded from the specified file or URL.

ShowImage

ANY QUESTION?